EE 641 – Homework 4: Attention and Transformers

Kuntian Tang
4855814938
kuntiant@usc.edu

October 27, 2026

1 Problem 1: Transformer for Multi-Digit Addition

(a) Metrics and training logs

All six runs use the same model, the same seed (641), and 2,400 Adam steps at batch 128. Test accuracy uses greedy decoding on the 2,000 test pairs.

Order	Train pairs	Test exact match	Test token acc.	steps_to_95
forward	1000	0.295	0.767	never
forward	2000	0.987	0.996	500
forward	4000	0.995	0.999	600
reversed	1000	0.9995	1.000	1250
reversed	2000	1.000	1.000	400
reversed	4000	1.000	1.000	450

Table 1: Test results and the first validation step with exact match ≥ 0.95.

(b) Visualizations

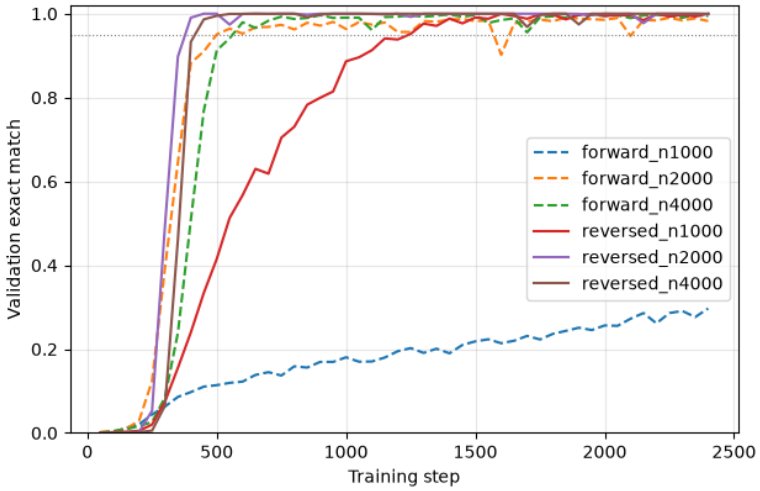


Figure 1: Validation exact match against step for the six runs.

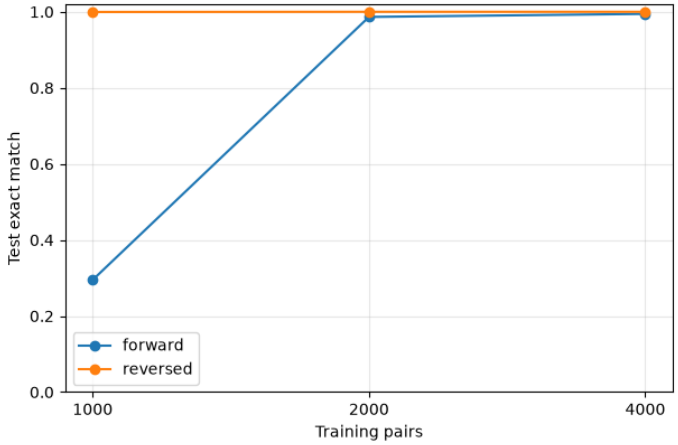


Figure 2: Test exact match against training size, per digit order.

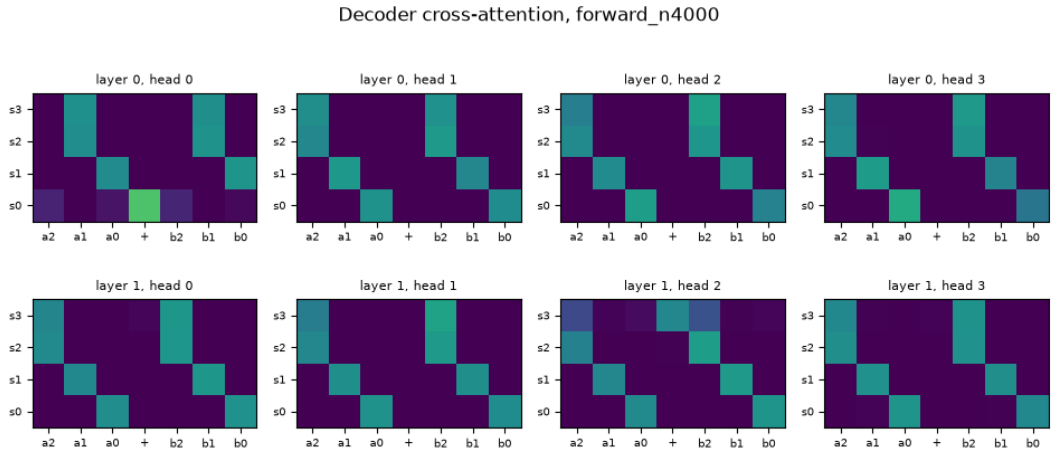


Figure 3: Mean decoder cross-attention, forward_n4000. Rows are output steps in emission order (s3 first).

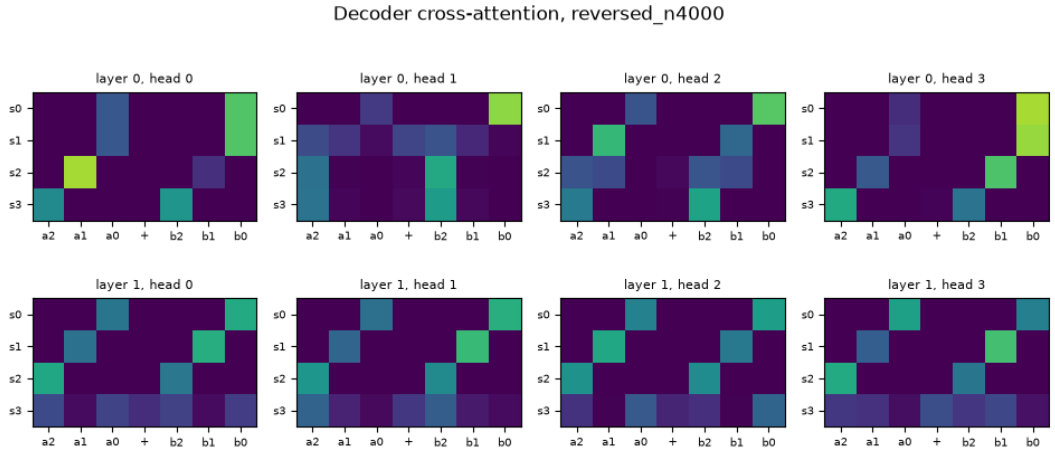


Figure 4: Mean decoder cross-attention, reversed_n4000 (s0 first).

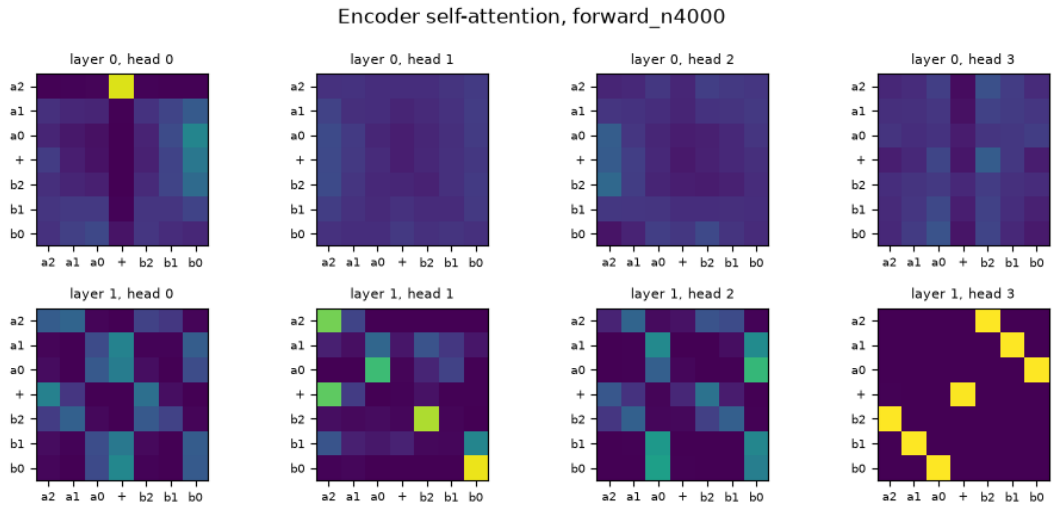


Figure 5: Mean encoder self-attention, forward_n4000.

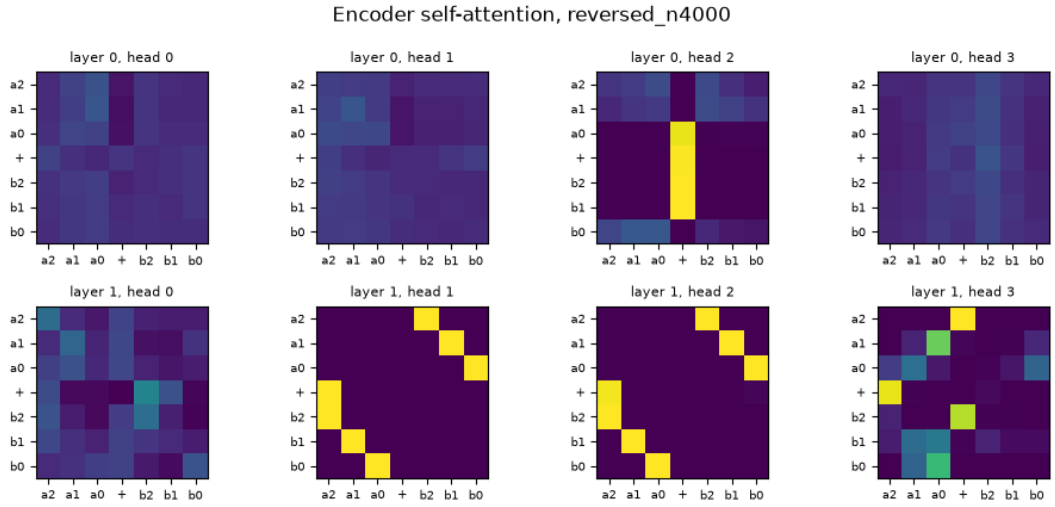


Figure 6: Mean encoder self-attention, reversed_n4000.

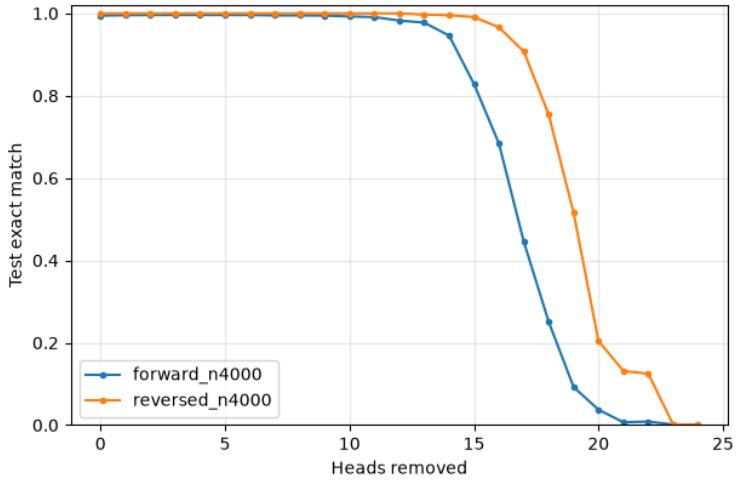


Figure 7: Greedy pruning: test exact match against the number of heads removed.

(c) Effect of digit order

The reversed order is better at every training size. The gap is largest at 1,000 pairs, where reversed reaches 0.9995 and forward reaches only 0.295. At 2,000 and 4,000 pairs both orders work. Reversed still reaches 95% sooner (400/450 steps against 500/600 steps), and it is exact on the test set where forward is not.

The cause is what output step j has to know. In the reversed order, step j emits place j, so it needs a_j, b_j, and the carry into place j. The carry is a function of a_{j-1}, b_{j-1} and the previous carry. The model has already emitted s_{j-1}, so it can see it through decoder self-attention. Each step is therefore a local computation on one digit pair, one lower pair, and its own history. This is the school algorithm, and few examples are enough to learn it.

In the forward order, the first output s_3 is the final carry. It depends on the whole chain of carries from the units place upward. The step s_2 also needs the carry into the hundreds place, which can depend on all lower digits (for example $a_1 + b_1 = 9$ passes a carry from the units). No earlier output contains this information. The network has to compute the full carry chain inside its two layers before it writes the first digit. With 1,000 pairs it memorizes the training set (training loss ≈ 0.02), but it does not learn this lookahead. Token accuracy of 0.77 shows that most single digits are correct. The errors are on the digits that need a carry.

(d) Cross-attention maps

Last decoder layer. In both models every head in layer 1 places about half its weight on a_p and half on b_p, where p is the place being emitted. In the forward model, s3 and s2 both read (a_2, b_2), s1 reads (a_1, b_1), and s0 reads (a_0, b_0). In the reversed model, s0 reads (a_0, b_0), s1 reads (a_1, b_1), and s2 reads (a_2, b_2). The reversed step s3 is the bare final carry and has no digit pair of its own. Its layer-1 attention is diffuse, so it gets this information from the decoder history and the layer-0 reads.

Where the carry is read (layer 0). The layer-0 maps show a second pattern, which reads the place *below* the output place. Forward, layer 0 head 0: s3 and s2 attend to (a_1, b_1), and s1 attends

to (a_0, b_0). These are exactly the inputs that decide the carry into the hundreds and tens places. Reversed, layer 0: head 0 sends s1 to (a_0, b_0) (the carry into place 1), and s2 to a_1 (the carry into place 2). Head 3 sends s0 and s1 to b_0, and head 2 sends s1 to (a_1, b_1). So the carry into place j is read in the first decoder layer from place $j-1$. The digit pair of place j is then added in the last layer.

The forward model has to read the lower place one step ahead and has no history that confirms the carry. A chain such as $\ldots 9 + \ldots 0$ with a carry from the units is invisible to the single place-below read. This explains the remaining forward errors.

(e) Head ablation and pruning

Single-head removal (intact: forward 0.995, reversed 1.000). Forward has six heads whose removal drops exact match below 0.95: `dec0_cross.h0` (0.35), `dec0_self.h0` (0.38), `dec0_self.h1` (0.48), `enc1_self.h3` (0.55), `dec0_self.h3` (0.59), and `dec1_cross.h1` (0.95). Reversed has three: `dec0_self.h0` (0.35), `dec0_self.h2` (0.40), and `dec0_cross.h3` (0.94). Removing any other single head costs almost nothing.

Greedy pruning. The reversed model loses 12 of 24 heads with *no* loss (still 1.000), and 16 heads while staying above 0.96. The forward model loses 9 heads at no cost and 12 heads while staying ≥ 0.98. After that, both curves fall steeply. A third to a half of the heads is enough. The reversed model needs fewer, which matches its simpler, local algorithm.

Do important heads survive pruning? Mostly yes, but not exactly. The last heads to be pruned in the forward model are `dec1_cross.h1`, `enc1_self.h3`, `dec0_self.h1/h0`, `dec1_cross.h0`, and `dec0_cross.h0`. Together with `dec1_cross.h3` and `dec0_self.h3`, these are the last eight heads, and all six heads that matter under single removal are among them. In the reversed model, `dec0_self.h0/h2` and `dec0_cross.h3` also survive almost to the end. Single-removal importance does not measure everything. The heads that survive longest also include `dec1_cross.h1` and `enc1_self.h1` in the reversed model, and both are harmless to remove alone. They are redundant with other heads, and they become critical only once the heads that duplicate them are gone. Single ablation therefore misses heads whose function is shared, and greedy pruning shows them.

2 Problem 2: Position Encodings and Length Extrapolation

(a) Metrics and training logs

Scheme	In range (0–63)	Beyond (64–127)	64–79	80–95	96–111	112–127
none	0.159	0.111	0.118	0.113	0.109	0.105
learned	1.000	0.074	0.085	0.071	0.067	0.072
sinusoidal	1.000	0.407	0.468	0.458	0.448	0.255
rotary	1.000	0.906	0.978	0.974	0.912	0.761

Table 2: Test accuracy over positions with a target (chance $= 1/16 = 0.0625$).

All three position schemes reach 100% in range after 5 epochs. Without position information the model cannot find position $i-7$. It does best at early positions, where few keys exist (0.23 in the

first bucket), and it falls toward chance as the sequence grows.

(b) Visualizations

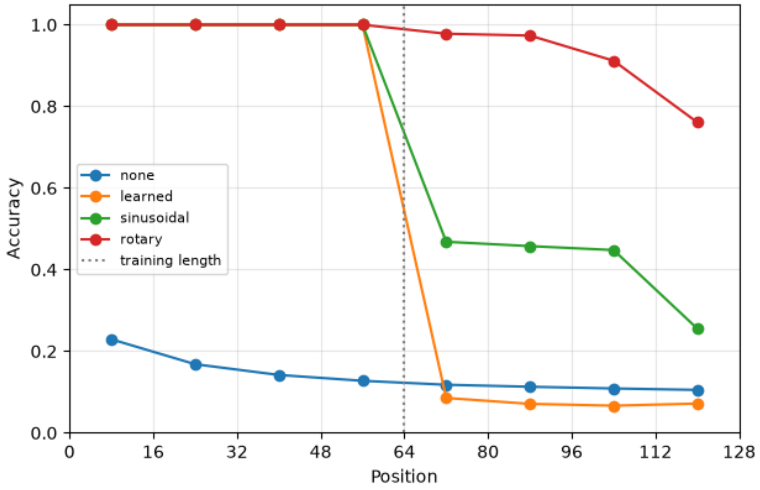


Figure 8: Accuracy per 16-position bucket. The dotted line is the training length 64.

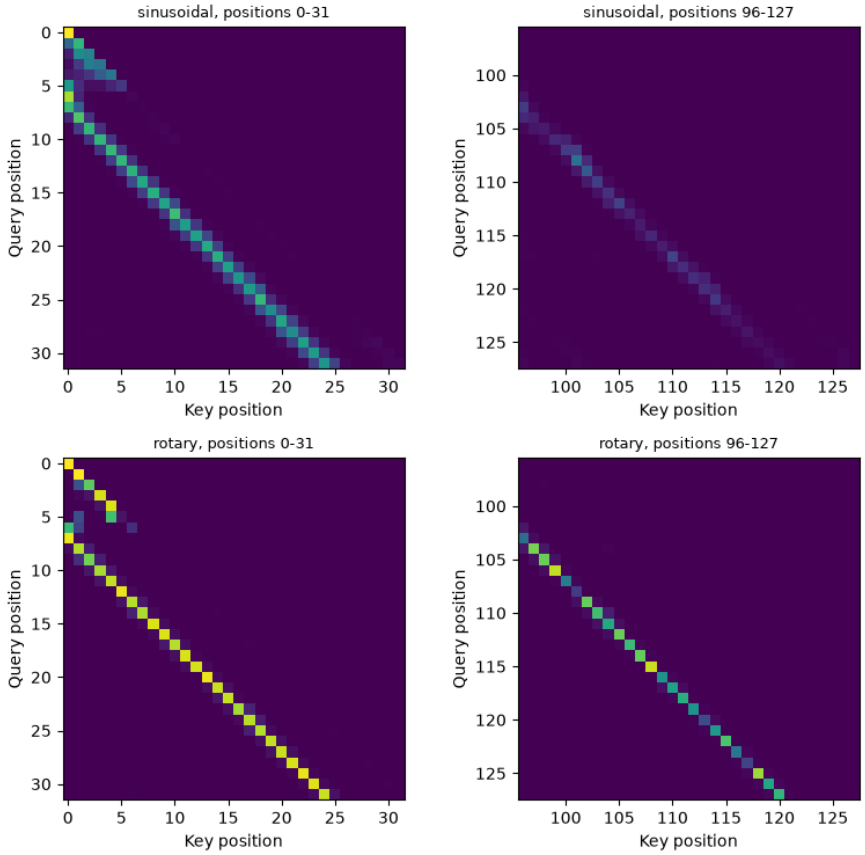


Figure 9: Attention of one test sequence, positions 0–31 and 96–127.

(c) The learned table beyond the training length

The learned table falls to 0.074, which is about chance and below the no-position model. Training sequences have 64 positions, so the embedding rows 64–127 never appear in a forward pass and never receive any gradient. They keep their random $\mathcal{N}(0, 1)$ initialization. At test time these random vectors are added to the token embeddings. The query/key projections were trained to recognize the specific learned rows 0–63, and they produce meaningless scores for the random rows. The "$i - 7$" pattern cannot transfer, because the table has no structure that relates row 70 to row 63.

(d) Encoding checks

Check	Value
`shift_rotation_max_error`	3.6×10^{-6}
`rotary_score_shift_error`	2.9×10^{-6}
`sinusoidal_score_shift_error`	9.39
`rotary_vs_pairwise_error`	8.6×10^{-6}

Shift rotation. By the angle-addition identities, each pair satisfies $\begin{bmatrix}\sin(p+s)\theta \\ \cos(p+s)\theta\end{bmatrix} = \begin{bmatrix}\cos s\theta & \sin s\theta \\ -\sin s\theta & \cos s\theta\end{bmatrix}\begin{bmatrix}\sin p\theta \\ \cos p\theta\end{bmatrix}$. The error is at float32 precision, so $\mathrm{PE}[p+s] = R_s\mathrm{PE}[p]$ holds exactly: an offset is a fixed linear map that does not depend on p. In principle, a linear attention layer can therefore express "look k back" with the same weights at every position.

Score shift. The rotary score depends only on $p_q - p_k$: $(R_{p_q}q)^\top(R_{p_k}k) = q^\top R_{p_k-p_q}k$. The error is at float32 precision. The sinusoidal score changes by 9.4 when both positions move. The reason is that $(q+\mathrm{PE}_{p_q})^\top(k+\mathrm{PE}_{p_k})$ contains the cross terms $q^\top\mathrm{PE}_{p_k}$ and $\mathrm{PE}_{p_q}^\top k$, and these depend on *absolute* position. In the model, the embedding also passes through W_q and W_k, which mix the content and the position. The relative structure of $\mathrm{PE}_{p_q}^\top\mathrm{PE}_{p_k}$ is therefore not preserved after the projections. The pairwise check confirms that the vectorized rotary implementation equals explicit per-pair 2×2 rotations.

Connection to the accuracies. Rotary attention scores are invariant to translation by construction. A pattern learned at positions 0–63 is the same function at 64–127, so accuracy stays at 0.97–0.98 for 32 positions past training. It decays (0.76 at 112–127) for a different reason. The score depends only on $p_q - p_k$, but a query at position 120 also scores keys at relative distances 64–120, and no training sequence contained these distances. Their scores are uncontrolled, and some of these distractors take softmax mass away from key $i-7$. The number of such keys grows with position. The sinusoidal encoding is a smooth, structured function of position, so it extrapolates partly (0.41, against 0.07 for learned). However, the absolute cross terms change as the position grows, and accuracy falls as the test position moves away from the training range.

(e) Attention maps

In positions 0–31, both models put their attention on a sharp diagonal at key $i-7$. Rotary is sharper (bright, close to one-hot). Sinusoidal spreads mass over neighbouring keys. Both show a different pattern at queries 0–6, which have no target.

In positions 96–127, the rotary map keeps the same clean $i-7$ diagonal. The weights are slightly lower, but the attention sits on the correct key, which matches the 0.76–0.91 accuracy in these buckets. The sinusoidal diagonal is still roughly in the right place, which shows the partial extrapolation. However, it is faint and spread over several keys, and it blurs and fades toward position 127. The model often retrieves a neighbour of $i-7$, which matches accuracy below 0.5.

3 Problem 3: Decoder-Only Transformer for SMILES

(a) Metrics, logs, and samples

The files are `q3/results/metrics.json`, 13 training logs, and `samples_pre_lr0.001_w500_s4000_T1.txt` (1,000 samples at $T=1$).

(b) Visualizations

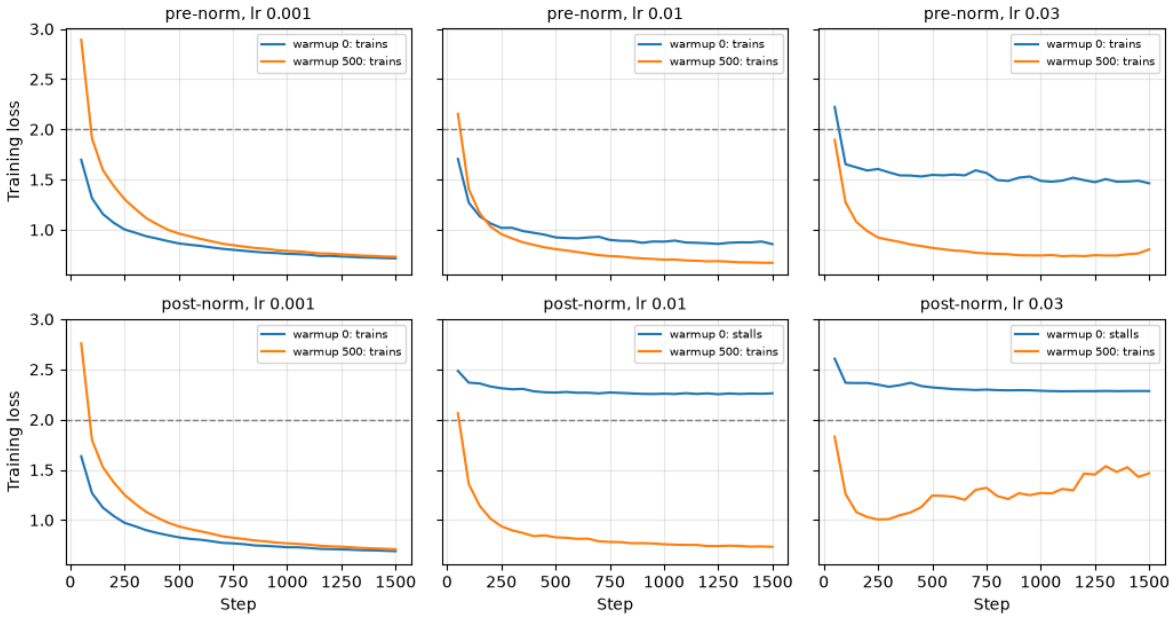


Figure 10: Training-loss curves of the sweep. The dashed line is the threshold of 2.0.

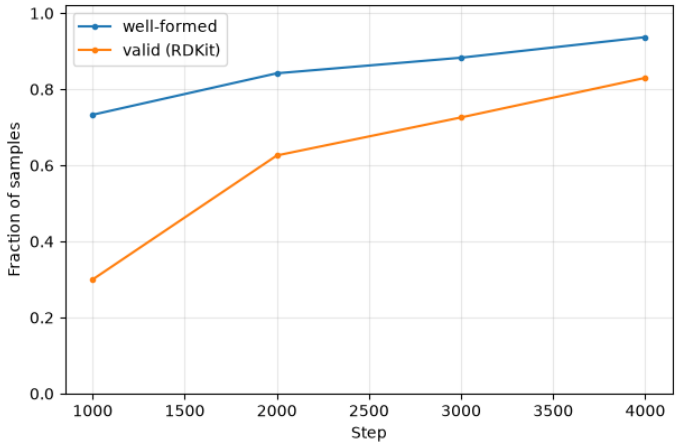


Figure 11: Well-formed and RDKit-valid fractions over training (1,000 samples, $T = 1$).

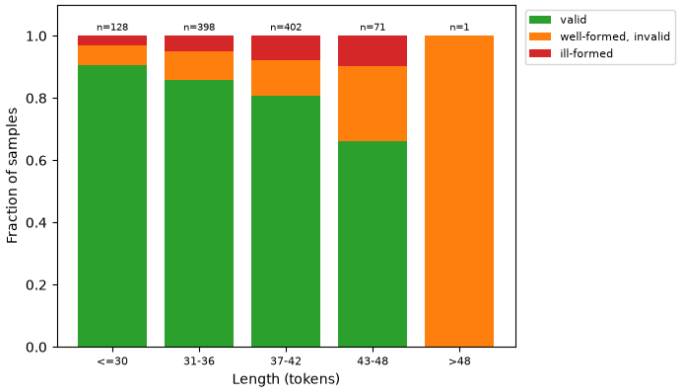


Figure 12: Outcomes by sample length (final model).

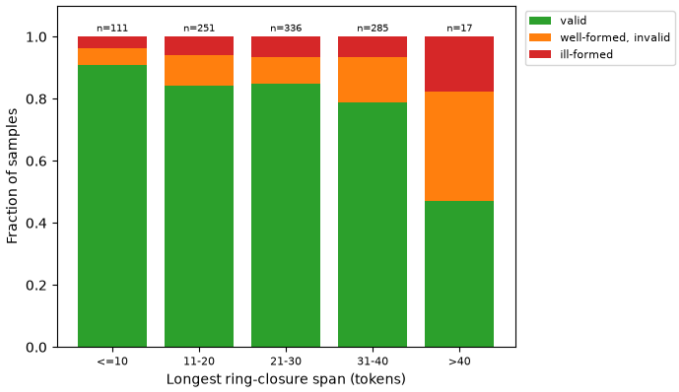


Figure 13: Outcomes by longest ring-closure span (final model).

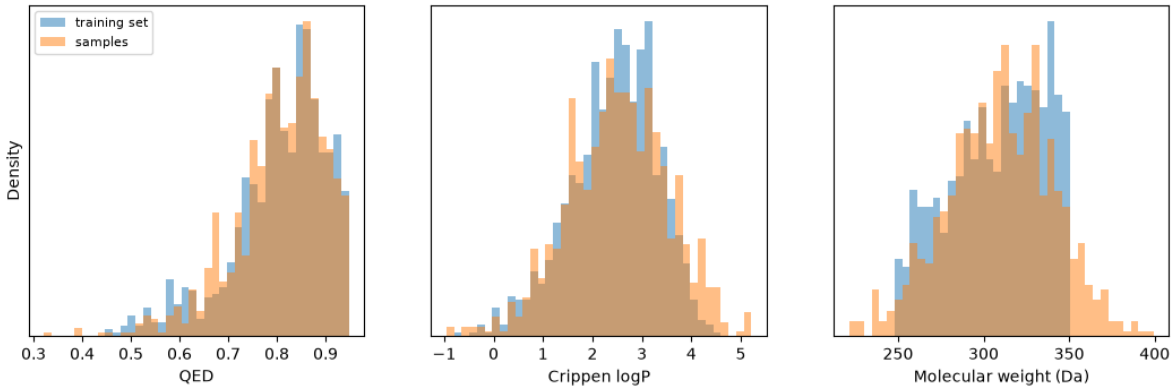


Figure 14: QED, logP, and MW of valid samples against 1,000 training molecules.

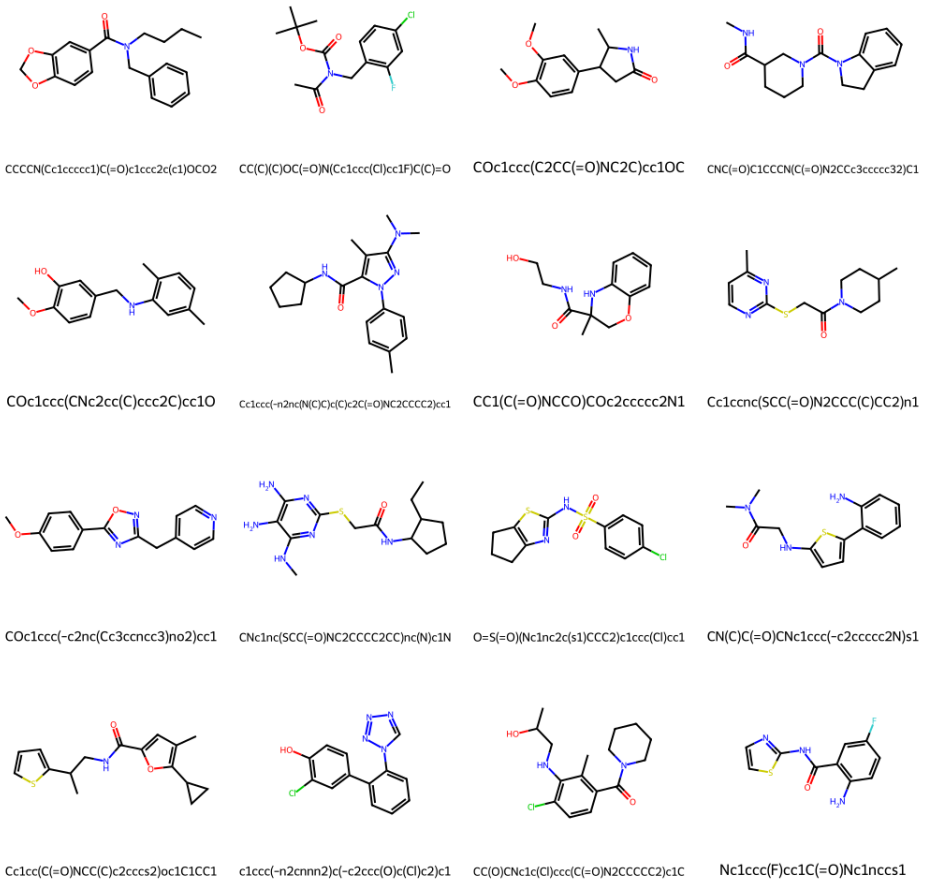


Figure 15: Sixteen valid samples.

(c) The sweep

Norm	Warmup	lr 10^{-3}	lr 10^{-2}	lr 3×10^{-2}
pre	0	trains (0.686)	trains (0.809)	trains (1.431)
pre	500	trains (0.701)	trains (0.648)	trains (0.757)
post	0	trains (0.661)	**fails** (3.396)	**fails** (5.099)
post	500	trains (0.679)	trains (0.702)	trains (1.338)

Table 3: Verdict (final validation loss < 2.0) after 1,500 steps.

Failing runs. Post-norm without warmup at 10^{-2} and 3×10^{-2} fails. Its training loss drops in the first steps and then sits on a flat plateau at ≈ 2.27 for the rest of the run. This value is about the entropy of the unigram token distribution: the model outputs token frequencies and ignores the context. The total gradient norm collapses from ~ 1 to 0.06–0.2 and stays there, so the network is in a dead state it does not leave. Validation loss is even higher (3.4 and 5.1) and drifts upward. In eval mode (without dropout), the degenerate network is miscalibrated. Two other runs train but are much worse. Pre-norm at 3×10^{-2} without warmup plateaus at 1.43. Post-norm at 3×10^{-2}

with warmup reaches 1.0 at the peak of warmup and then degrades back to 1.34. Both are on the edge of stability.

(d) Early head gradient norms

Run	mean	max	Run	mean	max
pre 1e-3 w0	0.29	1.53	post 1e-3 w0	0.32	1.63
pre 1e-3 w500	0.85	1.58	post 1e-3 w500	0.78	1.67
pre 1e-2 w0	0.53	4.75	post 1e-2 w0	0.52	**6.19**
pre 1e-2 w500	0.47	1.55	post 1e-2 w500	0.48	1.63
pre 3e-2 w0	0.62	6.65	post 3e-2 w0	0.61	**6.14**
pre 3e-2 w500	0.46	1.53	post 3e-2 w500	0.51	1.63

Table 4: Head-weight gradient norm over the first 100 steps (mean of the 50-step window means; max of the window maxima).

The early *max* head gradient norm separates warmup from no warmup at a high learning rate. Every run with a large peak lr and no warmup has a spike of 4.7–6.7. Every warmup run and every lr-10^{-3} run stays near 1.5–1.7. This is the size of the shock that the first full-lr updates give to the network. The early norms do *not* separate the runs that train from the runs that fail. Pre-norm 3e-2 w0 has the largest spike (6.65) and trains. Post-norm 1e-2 w0 has a similar spike and fails. The means (0.3–0.85) are not related to the outcome at all, and they mostly reflect the learning-rate schedule. The head gradient measures how violent the start is, but not whether the architecture can absorb it. Absorbing it depends on where the LayerNorm sits.

(e) Largest stable learning rate and the mechanism

	no warmup	warmup 500
pre-norm	3×10^{-2}	3×10^{-2}
post-norm	10^{-3}	3×10^{-2}

Pre-norm trains at every learning rate in the sweep, with or without warmup. Post-norm without warmup fails at 10^{-2} and above, and with warmup it trains up to 3×10^{-2} (but degrades there).

Mechanism. In pre-norm, $x_{l+1} = x_l + F(\mathrm{LN}(x_l))$, so the residual stream is an identity path from the embedding to the final LayerNorm and the head. The gradient into every layer contains an identity term $\partial x_L / \partial x_l = I + \ldots$, which does not grow or vanish with depth. A large update to one sublayer changes only its additive contribution. Each sublayer sees a normalized input, whatever the size of the stream.

In post-norm, $x_{l+1} = \mathrm{LN}(x_l + F(x_l))$, so every residual sum passes through a LayerNorm. The gradient to lower layers is multiplied by the Jacobian of each LayerNorm. That Jacobian is scaled by $1/\sigma$ of the sum and projects out the mean and the radial direction, and in a deep stack these factors compound. At initialization, the gradient is concentrated in the top layers and near the head, so with a large learning rate the first full-size Adam steps make the top sublayers dominate

the sum. LayerNorm then divides the residual signal by the larger σ, and the token and context information carried in the residual is renormalized away. The network falls into the context-free unigram solution, and the gradients through the normalization become tiny, which is the plateau with gradient norm ≈ 0.1. Warmup keeps the first few hundred steps small while Adam's second-moment estimates settle, so the layers grow together and the residual path survives. Pre-norm does not need this, because its identity path cannot be normalized away.

(f) Validity over training

Step	Well-formed	Valid	Unique	Novel
1000	0.733	0.300	1.000	1.000
2000	0.842	0.626	1.000	0.998
3000	0.883	0.726	1.000	0.996
4000	0.937	0.830	1.000	0.995

Both fractions rise monotonically. The model learns syntax first: at step 1000, 73% of samples are already well-formed, but only 30% are valid molecules. The gap between the two (0.43 at step 1000, 0.11 at step 4000) closes more slowly than the ill-formed fraction falls, so learning the chemistry lags behind learning the grammar.

Well-formed but invalid. The parentheses, rings, and brackets in these strings are balanced, but RDKit rejects the chemistry. Examples from the samples:

- `CC(=O)(CC(=O)N1CCc2sccc2C1)c1ccccc1`: the carbonyl carbon has =O plus three more bonds (valence 5).
- `Cc1cc(C)c(S(=O)(=O)N(CC(C)C)C(=O)N2CCCCC2)c(=O)c1C`: an exocyclic `c(=O)` in a ring of six aromatic carbons. The aromatic system cannot be kekulized.
- `CC(=O)c1ccc(NC(=O)c2nc(C(F)F)c(C)n2)cc1C`: a five-membered aromatic ring `c2nc..c(C)n2` with no `[nH]`. The heteroaromatic ring needs an explicit hydrogen to be kekulizable.

The ill-formed samples have syntax errors, for example a misplaced ring digit (`...N(S(=O)(=O)oc1)C1`: inside a branch, `oc1` closes the ring that `C1` opened, so the final `C1` opens a new ring that never closes), or ring digits opened and never closed (`Cc1nnc(NC23CCN(...)CC2)s1` leaves ring 3 open). A well-formed string is a grammatical sentence. A valid molecule also obeys valence and aromaticity rules, which are non-local and depend on counting bonds around an atom and electrons around a ring.

(g) Failure breakdowns

Length	≤ 30	31–36	37–42	43–48	> 48
n	128	398	402	71	1
valid	0.91	0.86	0.81	0.66	0
ill-formed	0.03	0.05	0.08	0.10	0
Span	≤ 10	11–20	21–30	31–40	> 40
n	111	251	336	285	17
valid	0.91	0.84	0.85	0.79	0.47
ill-formed	0.04	0.06	0.07	0.07	0.18

Validity falls steadily with length (0.91 down to 0.66), and both failure types grow. It also falls with the longest open ring span: it is about flat at 0.84 up to 30 tokens, and then it drops to 0.79 and 0.47 above 40. The ill-formed fraction grows from 0.04 to 0.18. The model finds long-range bookkeeping hard. It has to remember that a ring digit is open and close it with the matching partner, together with the valence and aromaticity state of the ring, across a long gap that contains branches. Long strings contain more and longer such dependencies and give more chances for one error. The local grammar is almost solved. Long-range consistency is the problem.

(h) Temperature

T	Well-formed	Valid	Unique	Novel
0.7	0.986	0.935	0.998	0.998
1.0	0.937	0.830	1.000	0.995
1.3	0.800	0.588	1.000	1.000

Lowering T sharpens the distribution toward high-probability, safe continuations: validity rises to 0.94 at $T = 0.7$, and it falls to 0.59 at $T = 1.3$. In this 100k-molecule, drug-like space, novelty and uniqueness stay very high at every temperature (0.995 or more). At $T = 0.7$, uniqueness is 0.998: a few duplicates appear, which is the first sign of mode collapse. The trade is real, but here it is lopsided. Lower temperature buys a large gain in validity for a very small loss in diversity. Higher temperature gives only a marginal gain in novelty, at the cost of a large loss in validity. The number of valid, novel molecules per 1,000 samples is ≈ 930 at $T = 0.7$, 820 at $T = 1$, and 590 at $T = 1.3$. The property distributions of valid samples at $T = 1$ closely match the training set for QED, logP, and molecular weight.